Машинное обучение

Лекция 3: Логистическая регрессия и градиентный спуск

Дьулустан Никифоров

Кафедра ИТ
Северо-Восточный Федеральный Университет

Весна 2026

Задача оптимизации в ML

Алгоритм персептрона для задачи линейной классификации был придуман напрямую (как будто «из воздуха»), свойства которого изучали уже потом.

В первой лекции «Введение в ML» мы рассмотрели общий подход к решению задачи **supervised learning (обучение с учителем)**, и вывели, что она сводится к оптимизационной задаче вида:

$$\Theta^* = \arg\min_{\Theta} J(\Theta),$$

где оптимизируемая функция $J(\Theta)$ имела вида

$$J(\Theta) = \left(\frac{1}{n} \sum_{i=1}^{n} L(h(x^{(i)}; \Theta), y^{(i)}) \right)$$

Задача оптимизации в ML

В общем случае, Θ представляет под собой *все* параметры рассматриваемой модели.

В случае линейной классификации, $\Theta = (\theta, \theta_0)$.

$L(\cdot, \cdot)$ — функция потерь.
Мы брали функцию потерь **0-1**:

$$L(h(x;\Theta), y) = \begin{cases} 0, & \text{если } h(x;\Theta) = y \\ 1, & \text{иначе.} \end{cases}$$

Для линейной классификации
$h(x;\Theta) = h(x;\theta,\theta_0) = sign(\theta^T x + \theta_0)$.

Регуляризация

Проблема с вышенаписанной функцией $J(\Theta)$ в том, что она думает *только* об обучающей выборке!

По идее, мы хотим, чтобы наш классификатор работал хорошо для *будущих* данных, а не конкретно для обучающих данных.

Один способ сделать это — выбрать достаточно ограниченное множество гипотез (моделей), из которых обучающий алгоритм будет выбирать.

Иногда вдобавок к этому, можно напрямую добавить слагаемое в целевую функцию:

Регуляризация

$$J(\Theta) = \left(\frac{1}{n} \sum_{i=1}^{n} L(h(x^{(i)}; \Theta), y^{(i)}) \right) + \lambda R(\Theta)$$

Этот метод называется **регуляризацией**, а $R(\Theta)$ — это **регуляризатор (regularizer)**,

Обычно, в качестве регуляризатора берут $R(\Theta) = \|\Theta - \Theta_{prior}\|$, где Θ_{prior} берется из каких-то других соображений.

Линейные классификаторы: проблемы

Для наших линейных классификаторов $J(\Theta)$ имеет вид

$$J(\Theta) = \left(\frac{1}{n} \sum_{i=1}^{n} L(sign(\theta^T x^{(i)} + \theta_0), y^{(i)}) \right)$$

Проблема с этой целевой функцией в том, что задача её минимизации NP-трудная.

- Что означает NP-трудность этой задачи?
- Как думаете, почему так происходит?
- А что с персептроном не так?

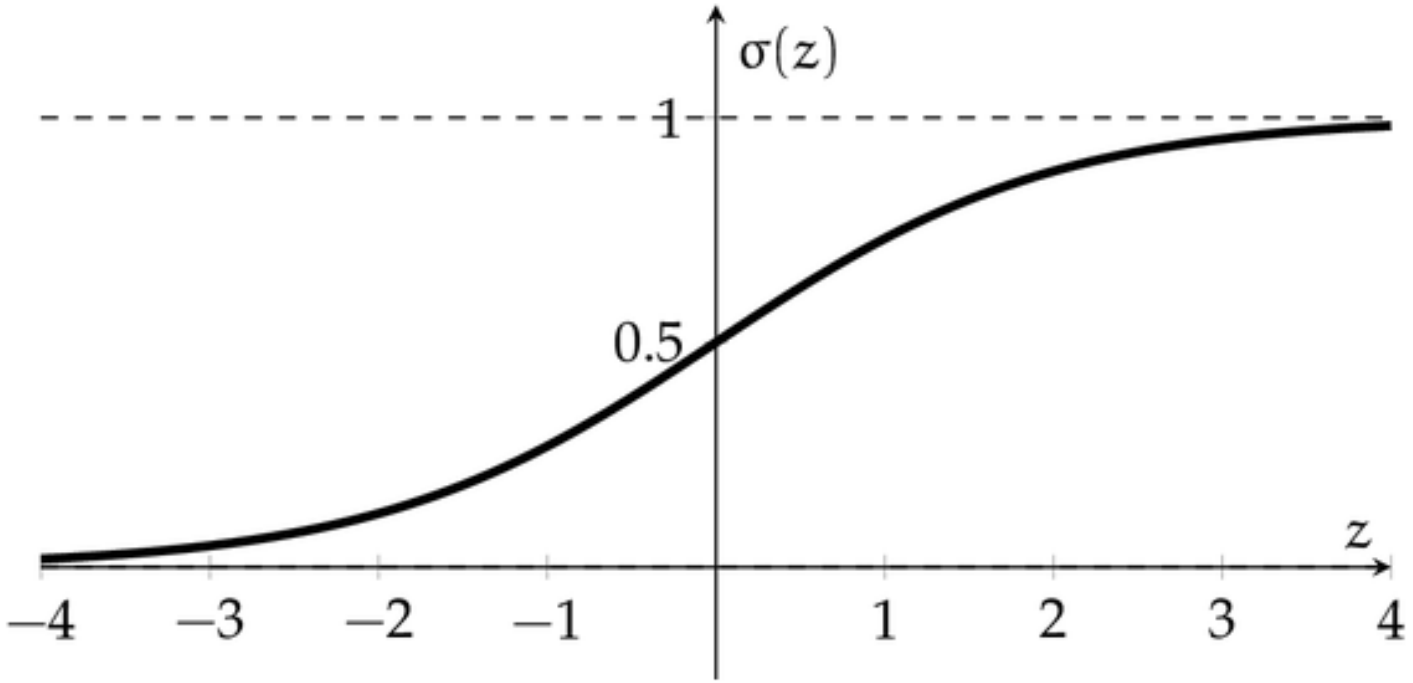
Линейные классификаторы: проблемы

- NP-трудная — нету алгоритмов, способных решить эту задачу за *полиномиальное* время.
- Функция потерь слишком «резкая» и неинформативная — не получается эффективно искать оптимальные (θ, θ_0).
- Персептрон решает эту задачу только в одном случае — когда существует (θ^*, θ_0^*) такая, что $J(\theta^*, \theta_0^*) = 0$ (линейная разделяемость!).

Линейные логистические классификаторы

Чтобы решить эти проблемы, мы теперь рассматриваем новый класс гипотез:
$h(x; \theta, \theta_0) = \sigma(\theta^T x + \theta_0)$ — **линейные логистические классификаторы**.

Здесь $\sigma(t) = \frac{1}{1+e^{-t}}$ — **логистическая функция**.



Линейные логистические классификаторы

Будем сокращенно называть их **LLC** (linear logistic classifier).

Однако, мы всё еще решаем задачу *бинарной* классификации, поэтому наши модели должны давать предсказания вида $+1$ и -1.

(!) Для LLC будем считать, что $y \in \{0, 1\}$ для удобства.

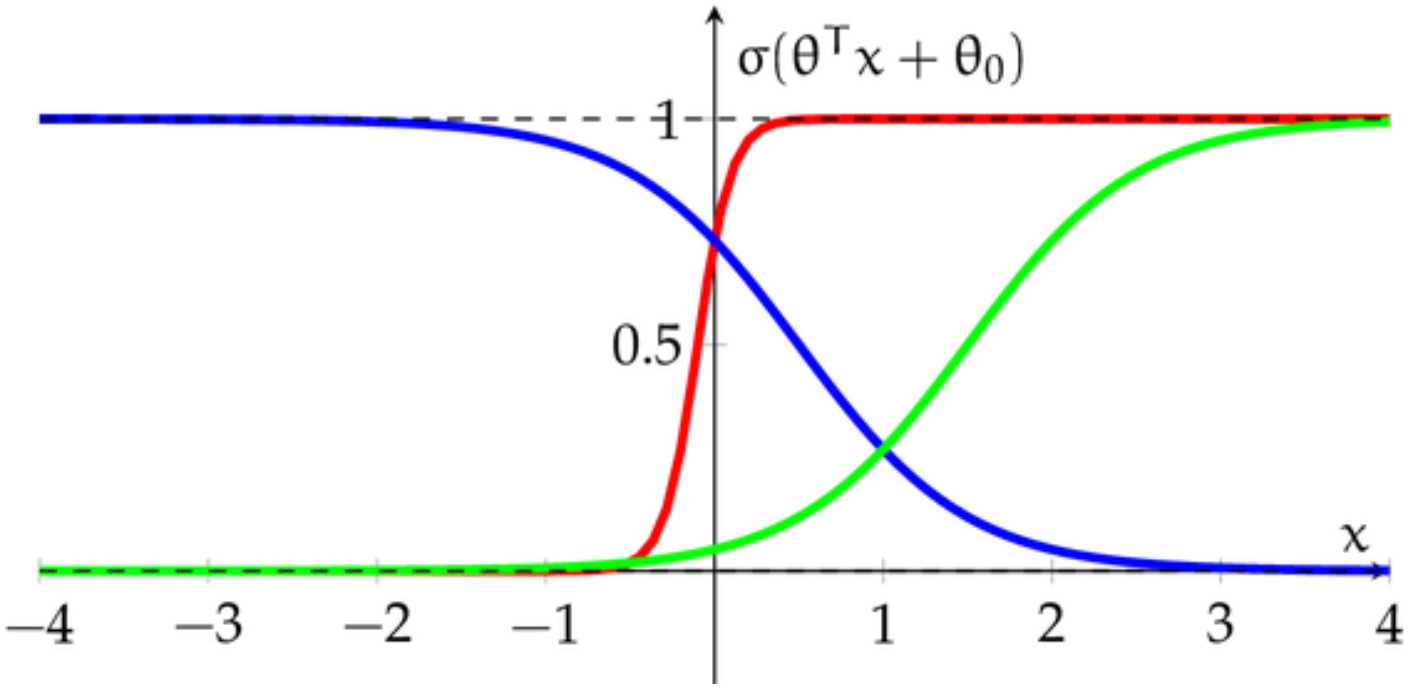
Будем считать, что модель $h(x; \theta, \theta_0) = \sigma(\theta^T x + \theta_0)$ предсказывает $y = 1$, если $\sigma(\theta^T x + \theta_0) > 0.5$, иначе $y = 0$.

0.5 называют **порогом предсказания (prediction threshold)**.

Порог предсказания могут делать и другим в некоторых случаях.

Линейные логистические классификаторы

Тут изображены графики трёх моделей
$\sigma(10x+1),\ \sigma(-2x+1),\ \sigma(2x-3)$.



Какой график соответствует каким моделям? Для каких значений x каждая из моделей предсказывает $+1$?

LLC: функция потерь

Когда необходимо сделать предсказание, мы смотрим на неравенство $\sigma(\theta^T x + \theta_0) > 0.5$.
Мы можем интерпретировать $\sigma(\theta^T x + \theta_0)$ как «вероятность», что у точки x класс 1, согласно гипотезе (θ, θ_0).

Обозначим $g^{(i)} = \sigma(\theta^T x^{(i)} + \theta_0)$ — guess гипотезы для точки $x^{(i)}$.

Если модель даёт $g^{(i)}$ для точки $(x^{(i)}, y^{(i)})$, то «вероятность», которую модель даёт правильному предсказанию $y^{(i)}$:

$$\begin{cases} g^{(i)}, & \text{если } y^{(i)} = 1 \\ 1 - g^{(i)}, & \text{иначе.} \end{cases}$$

LLC: функция потерь

Это то, что называют **likelihood** для точки $(x^{(i)}, y^{(i)})$.

Перепишем likelihood в виде произведения:

$$g^{(i)^{y^{(i)}}}(1 - g^{(i)})^{1-y^{(i)}}$$

Likelihood для всей выборки получается перемножением по всем точкам:

$$\prod_{i=1}^{n} g^{(i)^{y^{(i)}}}(1 - g^{(i)})^{1-y^{(i)}}$$

Мы хотим найти модель, у которой likelihood как можно больше!

LLC: функция потерь

Но максимизировать такое произведение вычислительно невозможно!

Какое монотонное преобразование для likelihood можно сделать, чтобы произведение превратить в что-то более приятное?

LLC: функция потерь

Но максимизировать такое произведение вычислительно невозможно!

Какое монотонное преобразование для likelihood можно сделать, чтобы произведение превратить в что-то более приятное?
Логарифм! При взятии логарифма от likelihood получается:

$$\sum_{i=1}^{n} \left(y^{(i)} \log g^{(i)} + (1 - y^{(i)}) \log(1 - g^{(i)}) \right)$$

Это то, что называют **log-likelihood**.
Мы хотим максимизировать log-likelihood.

LLC: функция потерь

Мы раньше определили оптимизационную задачу для задачи классификации как минимизацию целевой функции

$$J(\Theta) = \left(\frac{1}{n} \sum_{i=1}^{n} L(h(x^{(i)}; \Theta), y^{(i)}) \right),$$

поэтому возьмём в качестве функции потерь L функцию L_{nll}:

$$L_{nll}(g^{(i)}, y^{(i)}) = -\left(y^{(i)} \log g^{(i)} + (1 - y^{(i)}) \log(1 - g^{(i)}) \right)$$

$L_{nll}(g^{(i)}, y^{(i)})$ — **negative log-likelihood** функция потерь.

LLC: функция потерь

В итоге, наша задача нахождения LLC сводится к такой задаче минимизации:

$$J_{lr}(\theta, \theta_0) = \left(\frac{1}{n} \sum_{i=1}^{n} L_{nll}((\theta^T x^{(i)} + \theta_0), y^{(i)}) \right) + \lambda \|\theta\|^2 \to \min_{\theta, \theta_0}$$

Эту задачу оптимизации также называют **логистической регрессией (logistic regression)**.

Существует ОЧЕНЬ много методов оптимизации, которые можно было бы применять.
Мы сейчас изучим один из самых простых методов, называемый **градиентный спуск (gradient descent)**.

Градиентный спуск в 1D

Мы решаем задачу $J(\Theta) \to \min_\Theta$, где $\Theta \in \mathbb{R}$.
Пусть функция J дифференцируема т.е. у неё существует производная $J'(\Theta)$.
Тогда алгоритм **градиентного спуска**:

$\textsc{1D-Gradient-Descent}(\Theta_{init}, \eta, f, f', \epsilon)$

$\Theta^{(0)} = \Theta_{init}$
$t = 0$
repeat
    $t = t + 1$
    $\Theta^{(t)} = \Theta^{(t-1)} - \eta\, f'(\Theta^{(t-1)})$
until $|f(\Theta^{(t)}) - f(\Theta^{(t-1)})| < \epsilon$
return $\Theta^{(t)}$

Градиентный спуск в 1D

- Θ_{init} — исходная догадка о Θ;
- η — *длина шага* алгоритма;
- f — минимизируемая функция;
- f' — производная функции f;
- ϵ — параметр *точности*.

Условие остановки алгоритма можно и по-другому определять:

- Остановиться, когда $|f'(\Theta^{(t)})| < \epsilon$.
- Остановиться, когда $|\Theta^{(t)} - \Theta^{(t-1)}| < \epsilon$
- Остановиться после T итераций.

Градиентный спуск в 1D

Theorem

Если J — выпуклая функция, то для любой точности $\epsilon > 0$, найдется длина шага η такая, что градиентный спуск остановится на расстоянии не больше, чем ϵ от оптимального Θ.

Градиентный спуск

Пусть $\Theta \in \mathbb{R}^m$, $J : \mathbb{R}^m \to \mathbb{R}$.
Градиент функции J по отношению к Θ:

$$\nabla_\Theta J = \begin{bmatrix} \partial J / \partial \Theta_1 \\ \vdots \\ \partial J / \partial \Theta_m \end{bmatrix}$$

Алгоритм градиентного спуска такой же, но шаг 5 теперь:

$$\Theta^{(t)} = \Theta^{(t-1)} - \eta \nabla_\Theta J(\Theta^{(t-1)})$$

Логистическая регрессия: Градиентный спуск

Вернемся к нашей логистической регрессии:

$$J_{lr}(\theta, \theta_0) = \left(\frac{1}{n} \sum_{i=1}^{n} L_{nll}(g^{(i)}, y^{(i)}) \right) + \frac{1}{2}\lambda \|\theta\|^2 \to \min_{\theta, \theta_0}$$

$$L_{nll}(g^{(i)}, y^{(i)}) = -\left(y^{(i)} \log g^{(i)} + (1 - y^{(i)}) \log(1 - g^{(i)}) \right)$$

$$g^{(i)} = \sigma(\theta^T x^{(i)} + \theta_0)$$

Для удобства дифференцирования, к λ мы добавили коэффициент $\frac{1}{2}$.

Логистическая регрессия: Градиентный спуск

Если продиффиренцировать J_{lr} по θ и θ_0, то можно вывести:

$$\nabla_\theta J_{lr}(\theta, \theta_0) = \frac{1}{n}\sum_{i=1}^{n}(g^{(i)} - y^{(i)})x^{(i)} + \lambda\theta$$

$$\frac{\partial J_{lr}(\theta, \theta_0)}{\partial \theta_0} = \frac{1}{n}\sum_{i=1}^{n}(g^{(i)} - y^{(i)})$$

$\nabla_\theta J_{lr}(\theta, \theta_0)$ имеет размерность $d \times 1$,
$\frac{\partial J_{lr}(\theta,\theta_0)}{\partial \theta_0}$ — скаляр.

Логистическая регрессия: Градиентный спуск

LR-GRADIENT-DESCENT$(\theta_{init}, \theta_{0init}, \eta, \epsilon)$

$\theta^{(0)} = \theta_{init}$

$\theta_0^{(0)} = \theta_{0init}$

$t = 0$

repeat

$\quad t = t + 1$

$\quad \theta^{(t)} = \theta^{(t-1)} - \eta \left(\frac{1}{n} \sum_{i=1}^{n} \left(\sigma \left(\theta^{(t-1)^T} x^{(i)} + \theta_0^{(t-1)} \right) - y^{(i)} \right) x^{(i)} + \lambda \theta^{(t-1)} \right)$

$\quad \theta_0^{(t)} = \theta_0^{(t-1)} - \eta \left(\frac{1}{n} \sum_{i=1}^{n} \left(\sigma \left(\theta^{(t-1)^T} x^{(i)} + \theta_0^{(t-1)} \right) - y^{(i)} \right) \right)$

until $\left| J_{lr}(\theta^{(t)}, \theta_0^{(t)}) - J_{lr}(\theta^{(t-1)}, \theta_0^{(t-1)}) \right| < \epsilon$

return $\theta^{(t)}, \theta_0^{(t)}$

Стохастический градиентный спуск

Когда градиент имеет форму суммы многих похожих слагаемых, можно применить такую гениальную идею: вместо того, чтобы спускаться по всему градиенту, спуститься по рандомно выбранному слагаемому.

Это называется **стохастический градиентный спуск**.

В ML, градиент всегда выглядит как сумма по точкам обучающей выборки, поэтому стохастический градиентный спуск отлично подходит.

Стохастический градиентный спуск

Пусть целевая функция $f(\Theta) = \sum_{i=1}^{n} f_i(\Theta)$.
Тогда алгоритм:

$\textsc{Stochastic-Gradient-Descent}(\Theta_{init}, \eta, f, \nabla_\Theta f_1, \ldots, \nabla_\Theta f_n, T)$

$\Theta^{(0)} = \Theta_{init}$
for $t = 1$ **to** T
    randomly select $i \in \{1, 2, \ldots, n\}$
    $\Theta^{(t)} = \Theta^{(t-1)} - \eta(t)\, \nabla_\Theta f_i(\Theta^{(t-1)})$
return $\Theta^{(t)}$